TP 7-Youssef AOUZAH

Mission 1

simple code implementation for the RSA encryption decryption

```python
#challenge 1.1
p = 5
q = 11
e = 3
C = 23

n=p*q
phi=(p-1)*(q-1)
d=pow(e,-1,phi)

#decryption
m=pow(C,d,n)
print(m)


#encryption

cipher=pow(m,e,n)
print(cipher==C)
```

Using direct RSA algorithm reveals the password 12

```
PS C:\Users\Youssef\Desktop\s3\4-infoSec\TP-7> & C:/Users/Youssef/AppData/Lo
on.exe c:/Users/Youssef/Desktop/s3/4-infoSec/TP-7/challenges.py
12
```

Mission 2

first 64 bytes of the image

```
youssef@bruhness:/mnt/c/Users/Youssef/Desktop/s3$ cd 4-infoSec/
youssef@bruhness:/mnt/c/Users/Youssef/Desktop/s3/4-infoSec$ xxd -l 64 classified.enc
00000000: 5361 6c74 6564 5f5f 575a b8cf 6019 ac39  Salted__WZ..`..9
00000010: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
00000020: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
00000030: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
youssef@bruhness:/mnt/c/Users/Youssef/Desktop/s3/4-infoSec$
```

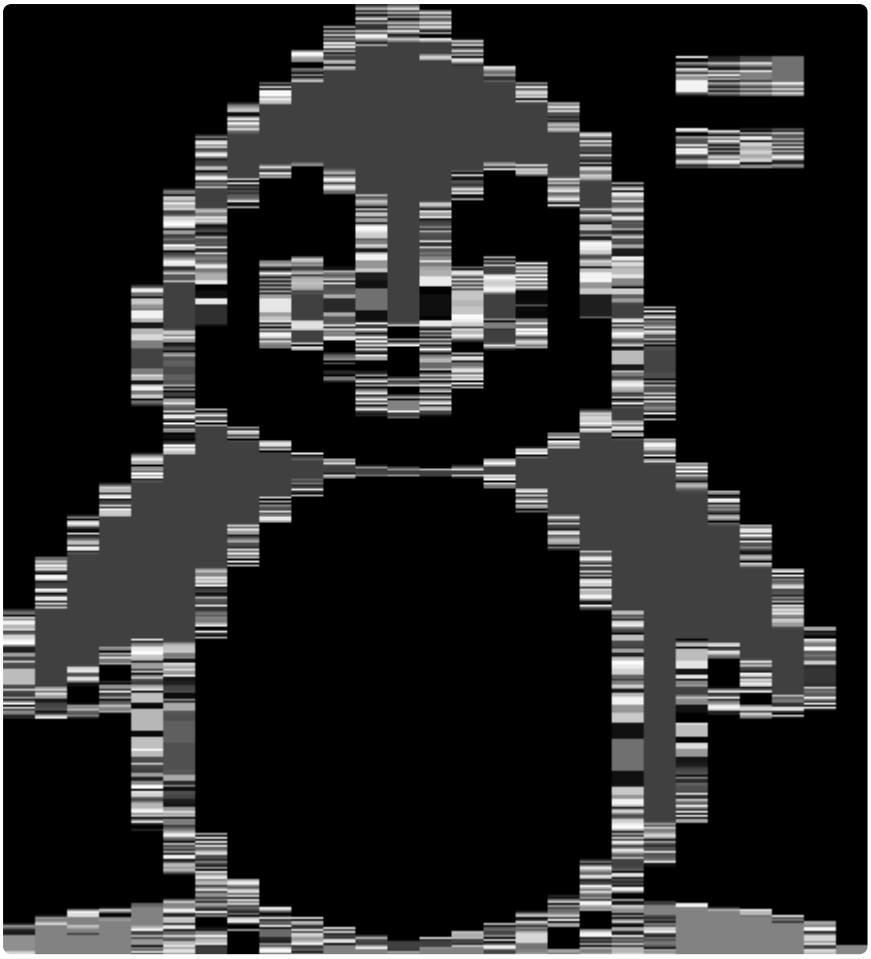
we can see that the first 16 bytes are dedicated to the signiture and the salt,the actual image bytes start at offset 0x10

thus:

The OpenSSL signature is the 8-byte ASCII string `Salted__` (`53 61 6c 74 65 64 5f 5f`).

The 8-byte salt immediately follows it: `57 5a b8 cf 60 19 ac 39`. Therefore, you must exclude the first **16 bytes** (offsets `0x00` to `0x0F`) before analyzing the actual AES ciphertext.

searching for patterns on the 16 bytes blocks leads to this image which gives a hint on the original one



using password 12 from mission 1 reveals the raw image

```
youssef@bruhness:/mnt/c/Users/Youssef/Desktop/s3/4-infoSec$ openssl enc -d -aes-256-ecb -pbkdf2 -p \
-in classified.enc -out recovered.raw
enter AES-256-ECB decryption password:
salt=575AB8CF6019AC39
key=103518D2C327E91924A746FA172211D62AC098CD2BA94E2270CC1D3612D6F877
youssef@bruhness:/mnt/c/Users/Youssef/Desktop/s3/4-infoSec$
```

the original image could then be retrieved


YOU
WIN